

SAVEETHA SCHOOL OF ENGINEERING (SIMATS)

Phases of a Compiler

Course: CSA14 – Compiler Design (Unit I: Introduction to Compiling)

Department of Computer Science and Engineering

 **Presenter:** V. Aravind

 **Register No:** 192321119

Overview & Architecture

Analysis Phase (Front-End)

Breaks up the source program into constituent parts and imposes a grammatical structure upon them.

- ✓ Includes Lexical, Syntax, and Semantic Analysis
- ✓ Generates Intermediate Representation (IR)
- ✓ Constructs and manages the Symbol Table
- ✓ Independent of target machine architecture

Synthesis Phase (Back-End)

Constructs the target program from the intermediate representation generated by analysis.

- ✓ Includes Code Optimization and Code Generation
- ✓ Converts IR into efficient machine code
- ✓ Handles register allocation & memory location
- ✓ Tailored directly to the target hardware architecture

</> Phase 1: Lexical Analysis

☰ Role of Lexical Analyzer

- ✓ Scans source code character by character from left to right
- ✓ Groups characters into meaningful sequences called **lexemes**
- ✓ Produces a stream of tokens in the format
- ✓ Strips out whitespace, tabs, and comments from source code
- ✓ Interacts directly with the Symbol Table for identifiers

600 × 400

Phase 2: Syntax Analysis

Parser Construction

- ✓ Receives token stream from the Lexical Analyzer
- ✓ Verifies tokens against grammatical rules of the language
- ✓ Builds a hierarchical structure called **Parse Tree** or **Abstract Syntax Tree (AST)**
- ✓ Detects structural syntax errors (e.g., missing semicolon or unbalanced parentheses)

AST & Key Outputs

- ✓ Node represents operators/statements; leaves represent operands
- ✓ Establishes operator precedence and associativity
- ✓ Passes verified AST structure to Semantic Analysis phase
- ✓ Reports syntax errors with exact position context

Semantic Analysis & IR



Semantic Analysis

Checks the parse tree for semantic consistency and type compatibility (e.g., ensuring integer cannot be assigned incompatible types).



Intermediate Code

Generates explicit, machine-independent representation such as **Three-Address Code (TAC)** or Quadruples.



Machine Independence

Decouples language parsing from target assembly, enabling easy retargeting to different CPU architectures (x86, ARM, RISC-V).

⚡ Phase 5: Code Optimization

🎯 Objectives of Optimization

Improves execution speed and reduces memory footprint without altering the program's functional output.

- ✓ **Constant Folding:** Computes compile-time constants ahead of execution
- ✓ **Dead Code Removal:** Eliminates code paths that can never be reached
- ✓ **Loop Optimization:** Moves invariant calculations outside loops

📊 Efficiency Gain



↔ Expression Transformation

Compiler Phase	Representation Format	Example Expression Transformation
Source Program	High-Level Statement	$position = initial + rate * 60$
Lexical Analyzer	Token Stream	$id1 = id2 + id3 * 60$
		$t1 = intofloat(60)$
Intermediate Code	Three-Address Code (TAC)	$t2 = id3 * t1$
		$t3 = id2 + t2$
Code Generator	Assembly Target Language	LDF R2, id3
		MULF R2, R2, #60.0
		STF id1, R1

>_ Example Code & Execution

</> Source Program (C Code)

```
#include

int main () {
    float rate = 5.0 ;
    float initial = 100.0 ;
    float position;

    // Statement compiled
    position = initial + rate * 60;

    printf ("Position = %.2f\n" , position);
    return 0;
}
```

► Compilation & Execution

```
# 1. Compile source program
$ gcc -O2 main.c -o position_calc

# 2. Run target executable
$ ./position_calc
Position = 400.00
```

- ✓ **Front-End:** Converts expression $5.0 * 60$ with implicit float conversion.
- ✓ **Optimizer:** Folds $5.0 * 60.0$ into 300.0 at compile-time.
- ✓ **Back-End:** Emits fast scalar addition for target CPU.

Symbol Table & Error Handling

Symbol Table Manager

- ✓ Central data structure accessed by all compiler phases
- ✓ Stores record for each identifier (variable, function name, label)
- ✓ Tracks key attributes: type, scope, memory address, and dimension
- ✓ Provides fast lookup and insertion capabilities during compilation

Error Handler

- ✓ Detects, reports, and recovers from errors across every phase
- ✓ **Lexical Errors:** Unrecognized character sequences
- ✓ **Syntax Errors:** Violation of grammar rules or missing tokens
- ✓ **Semantic Errors:** Type mismatches and undeclared variables

Questions & Answers

Thank you for your attention!

 Saveetha School of Engineering (SIMATS) | CSA14 - Compiler Design